KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

Parse Tree to Abstract Syntax Tree (AST) Generator

Submitted as a part of the course

Compiler Design

Submitted By

Student Name	Enrollment No.
Desai Hirva Alpeshbhai	202307020029

Program: B.Tech CSE
Semester: VII

Under the Guidance of

Dr. Bela Shrimali

Assistant Professor
Department of Computer Science & Engineering Technology

Karnavati University, Gandhinagar

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Overview of project
4. Technology used
5. Flow chart of the project
6. Screen Shots of the project
7. Results and Conclusion
8. Appendix

CHAPTER 1 – ABSTRACT

1.1 Project Summary

The Parse Tree to Abstract Syntax Tree Generator is a compiler design project developed to demonstrate the process of converting a source expression into a Concrete Syntax Tree and an Abstract Syntax Tree. The project provides a web-based user interface that allows users to enter arithmetic expressions and observe their internal syntactic representations. It is implemented using Python with the Flask web framework for processing, and HTML, CSS, and JavaScript for the browser interface, so the application runs locally and is used through a web browser.

The application performs lexical analysis to identify tokens such as identifiers, numeric constants, arithmetic operators, and parentheses. After tokenization, a recursive descent parser checks the grammatical structure of the input and generates a Concrete Syntax Tree. During the same parsing pass, a simplified Abstract Syntax Tree is produced by leaving out unnecessary grammar-related nodes while preserving the essential operators, operands, and expression relationships.

The interface displays the Concrete Syntax Tree (labelled "Parse Tree" on the web page) and the Abstract Syntax Tree using a hierarchical tree layout. It also provides token information, ASCII representations of both trees, status messages, and a summary of the number of tokens and the number of nodes in each tree. Users can select predefined examples or enter their own expressions to observe how different operators, parentheses, and unary minus influence the resulting syntax trees.

The primary objective of this project is to make the concepts of lexical analysis, syntax analysis, parse tree construction, and abstract syntax tree generation easier to understand through practical implementation and visual representation. The project serves as an educational tool for students studying compiler design and provides a foundation for understanding subsequent compiler stages, including semantic analysis, intermediate code generation, and optimization.

1.2 Purpose and Significance

The main purpose of the project is to demonstrate how a compiler processes a source expression and converts it into a structured representation that can be used by later stages of compilation. Understanding the difference between a Concrete Syntax Tree and an Abstract Syntax Tree is an important part of compiler design because these structures represent different levels of syntactic

information.

A Concrete Syntax Tree preserves the grammatical details used during parsing, including production-related nodes, parentheses, and other syntactic elements represented by the grammar. In contrast, an Abstract Syntax Tree focuses on the meaningful structure of the expression, representing operators and operands while eliminating unnecessary grammatical details. This transformation produces a more compact representation that is easier to process during subsequent compiler operations.

The project is significant because it combines theoretical concepts with a working web application. Instead of examining syntax trees only through textbook diagrams, users can enter expressions and immediately observe their corresponding tree structures. For example, an expression involving addition and multiplication demonstrates operator precedence, while expressions containing parentheses illustrate how grouping changes the structure of the resulting AST.

The application also helps users understand the role of lexical analysis and recursive descent parsing in the compilation process. By displaying the identified tokens and reporting syntax errors, it provides insight into how a compiler recognizes individual symbols and verifies whether an expression follows the supported grammar. The node-count summary further illustrates how the Abstract Syntax Tree is more compact than the Concrete Syntax Tree.

Overall, the project provides a practical learning environment for compiler design, improves understanding of syntax tree construction, and demonstrates how a simple web interface can make complex programming language concepts more accessible. Its modular implementation also allows future improvements, such as additional operators, assignment statements, semantic analysis, type checking, and intermediate code generation.

CHAPTER 2 – INTRODUCTION

2.1 Background

Compiler design is an important area of computer science that focuses on translating programs written in a high-level programming language into a form that can be understood and executed by a computer. During this translation process, a compiler performs several stages, including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and code generation. Each stage contributes to identifying the structure and meaning of the source program.

Two important structures used during syntax analysis are the Concrete Syntax Tree and the Abstract Syntax Tree. A Concrete Syntax Tree represents the grammatical structure of the input according to the rules of a grammar. It generally includes the details required to show how the input matches the grammar. An Abstract Syntax Tree provides a simplified representation of the program by focusing on its essential syntactic structure and omitting unnecessary grammatical details.

Understanding the relationship between these two tree structures is useful for learning how a compiler processes source code. The project, Parse Tree to Abstract Syntax Tree Generator, demonstrates this relationship through a web-based application. It allows users to enter an expression, view its tokens, observe its tree representation, and examine the corresponding abstract syntax structure.

The application provides a practical way to understand how source input is processed step by step. By presenting the results visually, it helps connect theoretical compiler concepts with an interactive implementation.

2.2 Problem Statement

Learning the internal stages of a compiler can be challenging for students because many compiler operations occur behind the scenes. Concepts such as tokenization, parsing, grammar rules, and syntax-tree construction are often explained theoretically, making it difficult to understand how they work together when processing an expression.

A written explanation alone may not clearly show how an input expression is divided into tokens, how its grammatical structure is determined, or how that structure can be simplified into an Abstract Syntax Tree. Students may also find it difficult to identify the relationships between operators, operands, and nested expressions without a visual representation.

The project addresses this learning challenge by providing an interactive visualizer for examining the processing of expressions. It accepts user input, performs tokenization, checks the expression through parsing, and displays tree representations to help users understand the structure of the input.

The application also provides supporting information, such as token details, status messages, and node counts. These features help users observe the processing results and understand the differences between the Concrete Syntax Tree and the Abstract Syntax Tree.

2.3 Objectives

The main objective of the project is to develop a web-based application that demonstrates the transformation of an input expression into tree-based representations used in compiler design.

The specific objectives are:

To accept user input: Provide an interface where users can enter an expression for analysis.

To perform lexical analysis: Divide the input into tokens representing elements such as identifiers, numbers, and operators supported by the application.

To perform syntax analysis: Use parsing logic to check whether the expression follows the grammar supported by the application.

To generate a Concrete Syntax Tree: Represent the grammatical structure of the input expression.

To construct an Abstract Syntax Tree: Present a simplified tree that emphasizes the essential relationships between operators and operands.

To visualize tree structures: Display the generated trees graphically so users can examine their structure and relationships.

To provide analysis information: Show token details, status messages, node counts, and textual tree representations in the interface.

To support learning: Help students understand fundamental compiler-design concepts through practical observation and experimentation.

Through these objectives, the project aims to make the basic stages of expression analysis easier to understand. It serves as an educational demonstration of tokenization, parsing, and syntax-tree representation rather than a complete compiler for a general-purpose programming language.

CHAPTER 3 – OVERVIEW OF THE PROJECT

3.1 Project Description

The Parse Tree to Abstract Syntax Tree Generator is a web-based application developed using Python, Flask, HTML, CSS, and JavaScript. The main purpose of the project is to demonstrate how an input expression is processed and represented using a Concrete Syntax Tree and an Abstract Syntax Tree. It provides an interactive environment in which users can enter expressions and observe the results of lexical analysis, syntax analysis, and syntax-tree generation.

The application accepts arithmetic expressions containing numbers, identifiers, the operators +, -, *, and /, parentheses, and unary minus. For example, a user can enter an expression such as a + b * c. The application processes the input, identifies its tokens, checks its grammatical structure, and generates the corresponding tree representations.

The project consists of several connected components. The lexical analyzer identifies the individual tokens present in the input. The parser checks the arrangement of these tokens according to the supported grammar and constructs a Concrete Syntax Tree. At the same time, it builds an Abstract Syntax Tree that focuses on the essential structure of the expression.

The application also provides a browser interface for displaying the generated trees. Users can examine the tree structures visually, view token information, and inspect textual representations of both trees. A statistics section displays the number of tokens, the number of nodes in the Concrete Syntax Tree, and the number of nodes in the Abstract Syntax Tree.

The project is designed primarily for educational purposes. It demonstrates important concepts of compiler design in a simple and accessible manner, helping students understand the relationship between source expressions, grammatical structures, and abstract representations.

3.2 Working Principle

The application follows a sequence of processing steps to transform the user's input into tree representations. Each step performs a specific task and passes its result to the next stage.

First, the user enters an expression into the input field and clicks the Generate Trees button. The browser sends the expression to the Flask server, where the lexical analyzer divides it into tokens such as identifiers, numbers, arithmetic operators, and parentheses. Each token contains its type and value.

After tokenization, the parser examines the sequence of tokens. The project uses a recursive-

descent parsing approach to check whether the expression follows the grammar implemented in the application. The grammar supports arithmetic expressions, parentheses, and unary minus. It also defines the relationship between arithmetic operators so that multiplication and division are handled at a higher precedence level than addition and subtraction.

If the input satisfies the supported grammar, the parser constructs a Concrete Syntax Tree. This tree contains grammatical nodes such as Expression, Term, and Factor, along with the operators, operands, and parentheses that explain how the expression is organized according to the grammar.

As each grammar rule is recognized, the parser also builds the corresponding node of the Abstract Syntax Tree. Unnecessary grammatical nodes are left out, while important elements such as operators, identifiers, and numeric values are retained. Parentheses are used to control the structure of an expression during parsing, but they do not remain as separate nodes in the abstract representation.

Finally, the server sends the results to the browser in JSON format, and the interface displays them. The Concrete Syntax Tree and Abstract Syntax Tree are shown in separate panels. Their textual representations are also available, along with the token list and the node counts.

If an unexpected character or invalid token sequence is encountered, the application reports an error instead of presenting the input as successfully parsed. This allows users to observe how the application responds to both valid and invalid expressions.

3.3 Main Features and Functionalities

The project includes several features that support the analysis and visualization of expressions.

1. Expression Input

The application provides an input field where users can type an expression for analysis. It also includes example expressions that can be selected for testing. A Clear button allows users to remove the current input, and expressions can be processed with the Generate Trees button or by pressing Ctrl+Enter.

2. Lexical Analysis

The lexical analyzer identifies the individual elements of an expression and classifies them into token types. These include identifiers, numeric values, and operators, where the operator type covers the arithmetic symbols and parentheses. If an unsupported character is found, the analyzer reports its position in the input.

3. Syntax Analysis

The parser checks whether the input follows the grammar supported by the project. It uses recursive-descent parsing methods to recognize expressions, terms, and factors. The parser also checks for unexpected characters, missing operands, missing closing parentheses, and trailing tokens that prevent the expression from being accepted.

4. Concrete Syntax Tree Generation

The application generates a Concrete Syntax Tree that represents the grammatical structure of the input. The tree includes grammar-related nodes and the corresponding terminal elements. This representation helps users understand how the parser organizes an expression according to the grammar rules.

5. Abstract Syntax Tree Generation

The application produces an Abstract Syntax Tree alongside the Concrete Syntax Tree. The AST removes unnecessary grammatical details and emphasizes the meaningful relationships between operators and operands. This simplified structure is useful for understanding how compilers represent expressions internally.

6. Graphical Tree Visualization

The application displays both tree structures graphically. Nodes are arranged in a hierarchical layout, with parent nodes connected to their children. Separate panels make it possible to compare the Concrete Syntax Tree and Abstract Syntax Tree and observe the differences between them.

7. Token Display

A dedicated section displays the identified tokens, showing each token's type and value. This view provides a clear summary of the lexical analysis stage and helps users understand how the original expression is divided into individual components.

8. Textual Tree Representation

The application provides an additional view of both trees using an ASCII-based textual representation. This feature makes it possible to inspect the hierarchical structure without relying exclusively on graphical diagrams.

9. Processing Status and Summary

The interface displays a success or error message and provides a summary of the generated trees. The summary includes the number of tokens, the number of nodes in the Concrete Syntax Tree, and the number of nodes in the Abstract Syntax Tree. Comparing these values helps users understand how the abstract representation simplifies the original tree.

Overall, these features combine lexical analysis, recursive-descent parsing, syntax-tree construction, and graphical visualization into a single educational application. The project offers a practical demonstration of fundamental compiler-design concepts while remaining limited to the expression forms and grammar implemented in the application.

CHAPTER 4 – TECHNOLOGY USED

4.1 Python Programming Language

Python is the primary programming language used to develop the Parse Tree to Abstract Syntax Tree Generator. It is a high-level, interpreted programming language known for its readable syntax, flexibility, and support for structured programming. These characteristics make Python suitable for implementing compiler-design concepts and developing educational applications.

In this project, Python is used to implement the lexical analyzer, recursive-descent parser, tree-node structures, AST construction logic, node counting, and textual tree-rendering functions. Flask, a lightweight Python web framework, connects these components to the browser. It serves the main page and provides a /parse route that receives an expression, runs the analysis, and returns the results.

The project uses a Parser class to organize the recursive-descent parsing methods, along with functions for tokenization, node counting, and ASCII tree generation. Python's regular-expression module is used by the lexer to recognize numbers, identifiers, and operators in the input.

Python also provides built-in features for handling strings, lists, dictionaries, and exceptions. These features help the application process expressions, store tokens, construct hierarchical trees, and report errors when the input does not follow the supported grammar.

Another advantage of this approach is that the application needs only Python and Flask to run. This makes the project relatively easy to install, understand, and modify for educational experimentation.

4.2 Web User Interface

The user interface is built with HTML, CSS, and JavaScript and runs in a standard web browser. HTML defines the structure of the page, CSS controls its appearance and layout, and JavaScript handles user actions and displays the results returned by the server. The page is delivered by Flask as a template.

The interface allows users to enter expressions, select predefined examples, clear the input, and generate trees. Pressing Ctrl+Enter in the input box also starts the analysis, and a default example is analyzed automatically when the page loads.

Different sections of the page are dedicated to different types of information. The tree panels display the Concrete Syntax Tree and Abstract Syntax Tree. The lexical analysis section presents

the tokens, while the ASCII section displays textual representations of the generated trees. A grammar section lists the rules used by the parser.

The trees are drawn using nested HTML lists styled with CSS, which creates the nodes and the connecting lines between parents and children. Scrollable containers help users navigate tree structures that extend beyond the visible area. Node names are inserted using text content, so expression text is never interpreted as HTML.

The use of a web interface makes the project interactive and visually accessible. Instead of examining only program output in a terminal, users can explore the parsing process through a page that updates with each analysis.

4.3 Compiler Design Concepts and Parsing Techniques

The project applies several fundamental compiler-design concepts, including lexical analysis, syntax analysis, grammar-based parsing, Concrete Syntax Tree construction, and Abstract Syntax Tree generation.

Lexical Analysis: Lexical analysis is the process of dividing an input string into meaningful units called tokens. In this project, the lexical analyzer recognizes identifiers, numbers, and the operators and parentheses used in arithmetic expressions. Whitespace is skipped, and unsupported characters are reported with their position.

Syntax Analysis: Syntax analysis examines the sequence of tokens to determine whether it follows the grammar supported by the application. The project implements a recursive-descent parser, in which separate methods process different grammatical structures.

Recursive-Descent Parsing: Recursive-descent parsing is a technique in which parsing methods correspond to grammar rules and call one another to recognize expressions. The project uses this approach to process expressions, terms, and factors. The grammar structure also supports operator precedence by separating addition and subtraction from multiplication and division.

Concrete Syntax Tree: A Concrete Syntax Tree represents the grammatical structure recognized by the parser. It includes grammar-related nodes and terminal elements, helping users understand how the input is derived from the grammar.

Abstract Syntax Tree: An Abstract Syntax Tree provides a simplified representation of the expression. The project builds the AST during parsing by omitting unnecessary grammatical structures while preserving the essential relationships between operators and operands.

These concepts form the core processing mechanism of the application. Their implementation demonstrates how a compiler-related tool can analyze an expression and represent its structure in

a form suitable for further processing.

Code Snippet of lexical analyzer :

```
def tokenize(expression):
    tokens = []
    position = 0

    while position < len(expression):
        if expression[position:].isspace():
            break

        match = TOKEN_PATTERN.match(expression, position)
        if not match:
            raise ParseError(f"Invalid character at position {position + 1}")

        number, identifier, operator = match.groups()
        if number is not None:
            tokens.append({"type": "NUMBER", "value": number})
        elif identifier is not None:
            tokens.append({"type": "IDENTIFIER", "value": identifier})
        else:
            tokens.append({"type": "OPERATOR", "value": operator})

        position = match.end()

    tokens.append({"type": "EOF", "value": "EOF"})
    return tokens
```

4.4 Data Structures, Error Handling, and Visualization

The project uses several Python data structures and programming techniques to manage tokens, construct syntax trees, handle invalid input, and display results.

A token is represented using a dictionary containing its type and value. Token types distinguish identifiers, numbers, operators, and the end-of-input marker used internally by the parser.

The syntax trees are constructed using a node-based structure. Each node is a dictionary containing a name and, where applicable, a list of child nodes. This arrangement represents the hierarchical relationships between grammatical elements. A recursive counting function determines the total number of nodes in a tree, allowing the application to compare the sizes of the CST and AST.

The application also includes an ASCII tree-rendering function. It walks through the tree recursively and prefixes each node with connecting characters to produce a textual representation of the hierarchy. This provides an alternative way to inspect the generated structures.

Error handling is implemented through exceptions. If the lexer encounters an unsupported character, or if the parser encounters an invalid token sequence, a parse error is raised. The server catches the error and returns a message with an error status, which the page displays in a message box. Excessively nested input is also handled. This prevents invalid input from being presented as a successful parsing result.

For graphical visualization, the application uses HTML and CSS to arrange nodes and draw connections between parent and child nodes. Parse tree nodes, AST nodes, and leaf nodes are shown in different colours, and the two trees are displayed separately, making their differences easier to observe.

Together, these data structures, error-handling mechanisms, and visualization techniques support the project's main objective: demonstrating the transformation of an input expression into Concrete and Abstract Syntax Trees through an interactive interface.

CHAPTER 5 – FLOW CHART OF THE PROJECT

5.1 Project Workflow

The Parse Tree to Abstract Syntax Tree Generator follows a systematic workflow to process an input expression and display its corresponding tree representations. The workflow begins when the user enters an expression into the web interface. The application then performs lexical analysis, syntax analysis, tree construction, and result visualization.

Initially, the application checks whether the input field contains an expression. If the input is empty, an appropriate message is displayed to guide the user. If the input is available, the browser sends it to the server, where the lexical analyzer scans the expression and converts it into a sequence of tokens. Each token contains its type and value.

After tokenization, the parser examines the tokens according to the grammar supported by the application. The parser uses recursive-descent parsing to identify the grammatical structure of the expression. If an unexpected character or an invalid token sequence is encountered, the application reports an error and does not display the expression as successfully parsed.

When the input is valid, the parser generates a Concrete Syntax Tree. Together with it, the parser produces an Abstract Syntax Tree that leaves out unnecessary grammatical details while preserving the essential structure of the expression.

Finally, the application displays the generated trees in graphical and textual forms. It also presents token information, a status message, and a summary containing the token count and the node counts of both trees. The user can modify the expression and repeat the process to analyze another input.

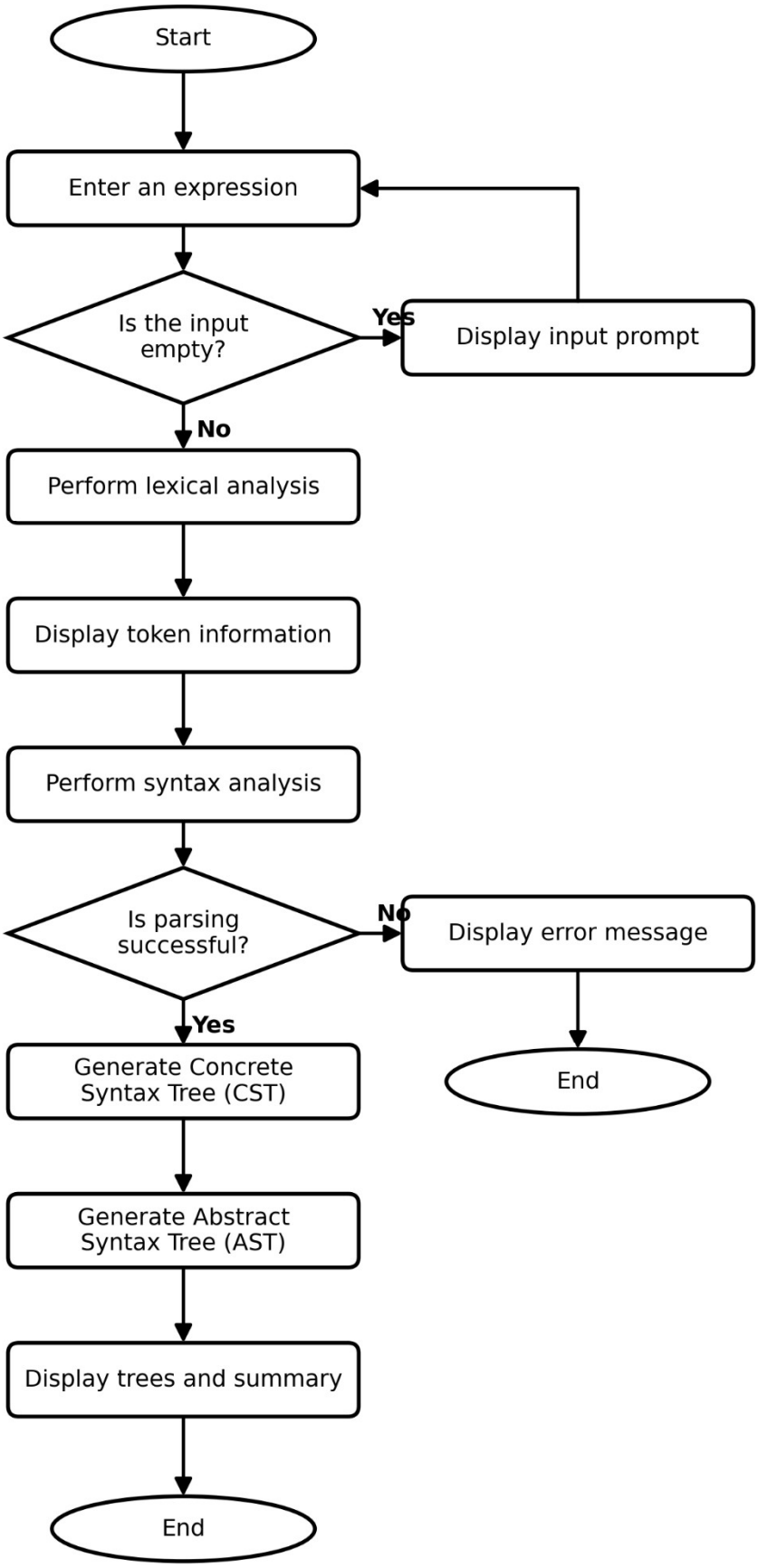

Start
Enter an expression
Is the input empty?
Yes
Display input prompt
No
Perform lexical analysis
Display token information
Perform syntax analysis
Is parsing successful?
No
Display error message
End
Yes
Generate Concrete Syntax Tree (CST)
Generate Abstract Syntax Tree (AST)
Display trees and summary
End

Figure 5.1: Project Workflow

5.2 Flow of Lexical and Syntax Analysis

Lexical analysis and syntax analysis are the two main processing stages performed before the syntax trees are displayed.

During lexical analysis, the input expression is scanned from left to right. Whitespace characters are skipped, while identifiers, numbers, operators, and parentheses are recognized as tokens. Numbers may contain a decimal part, and identifiers begin with a letter or underscore. If a character cannot be matched, an error reporting its position in the input is raised. An end-of-input token is added after the expression has been processed.

The resulting token sequence is passed to the syntax analyzer. The parser checks whether the tokens can be arranged according to the grammar implemented in the project. The grammar recognizes arithmetic expressions containing addition, subtraction, multiplication, division, identifiers, numeric values, parenthesized expressions, and unary minus.

The recursive-descent parser uses separate parsing methods to process different parts of the grammar. Expressions are processed through expression and term structures, while factors represent identifiers, numbers, negated factors, or parenthesized expressions. This organization supports the intended precedence of arithmetic operators, with multiplication and division handled within terms before addition and subtraction are handled at the expression level.

If parsing succeeds, the application produces the Concrete Syntax Tree and the Abstract Syntax Tree. If parsing fails, an error message is displayed and no new trees are generated for that unsuccessful analysis.

This sequence demonstrates how a compiler-related application converts a sequence of tokens into a structured representation while detecting errors in the input.

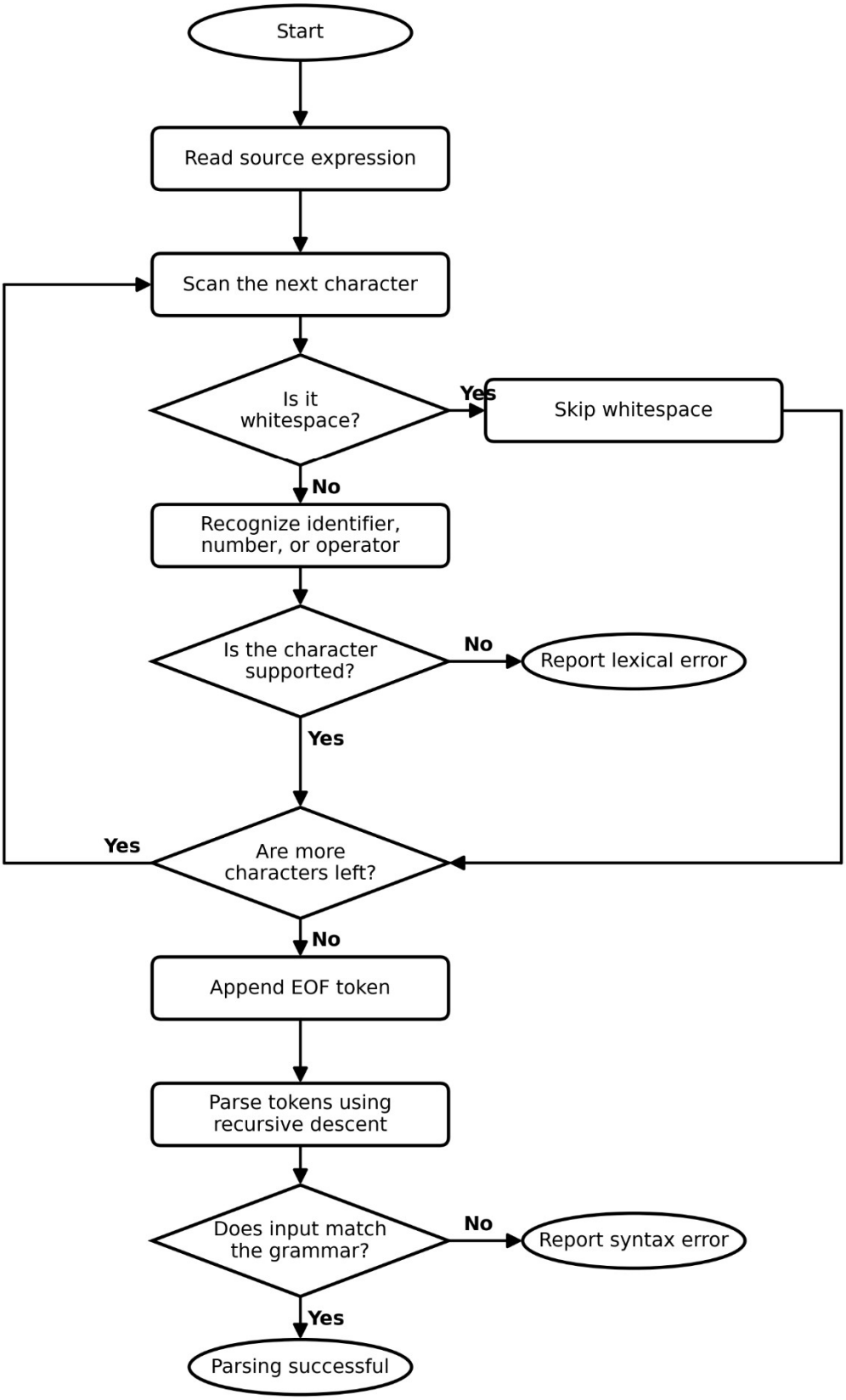


Figure 5.2: Lexical and Syntax Analysis Flow

5.3 Flow of Syntax Tree Generation and Output

After successful syntax analysis, the application proceeds to syntax-tree output and visualization. The Concrete Syntax Tree is created during parsing and contains the grammatical elements used to represent the input expression.

The Abstract Syntax Tree is built in the same parsing pass. As each grammar rule is recognized, the parser returns both its concrete node and a simplified abstract node. Grammar-related nodes that are not essential to the abstract structure are left out. Operators and operands are retained in a hierarchical arrangement that reflects their relationships within the expression.

For example, consider the expression a + b * c. The abstract representation places the addition operator at the root, with the identifier a on one side and the multiplication operation on the other. Beneath the multiplication operator are b and c, which preserves operator precedence. This makes the structure of the expression easier to understand.

Once both trees are available, the application calculates their node counts. It uses these values, together with the token count, to display the statistics summary in the interface.

The browser then presents the Concrete Syntax Tree and Abstract Syntax Tree in separate panels. Users can also inspect the token list and the ASCII representations of the trees. If the input is changed, the application processes the new expression and updates the displayed results. The output stage completes the project's processing cycle by presenting the results in forms that support comparison and understanding.

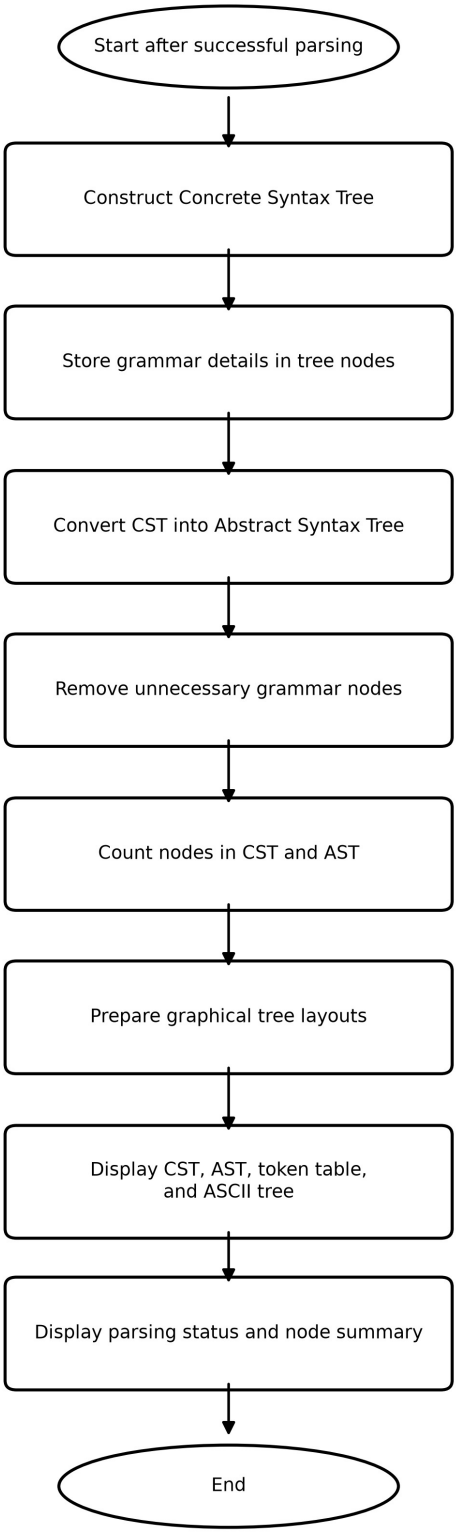


Figure 5.3: Syntax Tree Generation and Output Flow

CHAPTER 6 – SCREEN SHOTS OF THE PROJECT

6.1 Main Application Interface

The main application interface provides an interactive environment for entering expressions and observing their analysis. It is developed using Python, Flask, HTML, CSS, and JavaScript and is designed to present the different stages of expression processing in an organized manner.

The interface contains an input field where users can enter an expression. It also provides options to select predefined examples, clear the current input, and generate the trees. The page analyzes a default example when it loads, allowing users to see results immediately.

A message area displays the current result, indicating whether the expression has been processed successfully or whether an error has occurred. The interface also includes separate sections for graphical tree visualization, token information, and textual tree representations.

The main page brings the application's features together, making it easier for users to explore the relationship between an input expression and its syntax-tree representations.

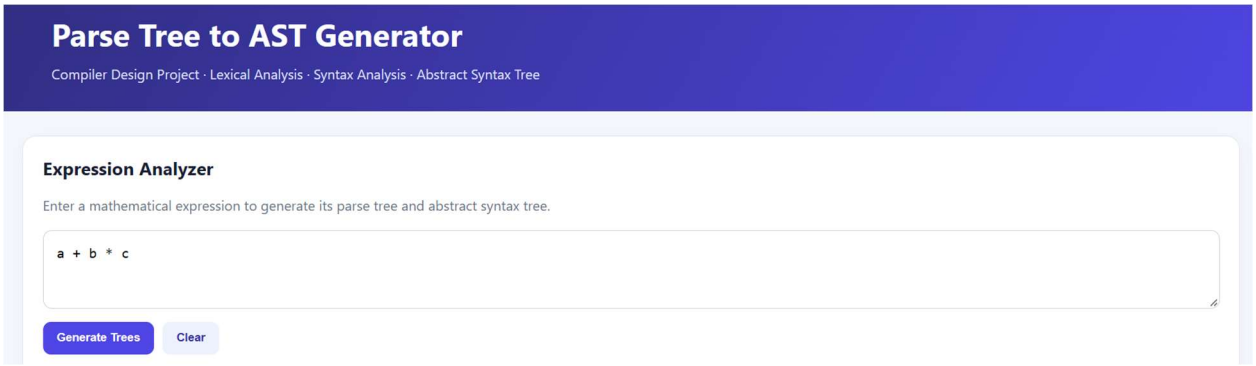


Figure 6.1: Main Application Interface

6.2 Concrete Syntax Tree and Abstract Syntax Tree Visualization

The graphical visualization section displays the Concrete Syntax Tree and Abstract Syntax Tree generated from the input expression. The two structures are presented in separate panels so that users can compare their organization.

The Concrete Syntax Tree, labelled Parse Tree on the page, illustrates how the expression follows the grammar implemented in the parser. It contains grammatical nodes and terminal elements that represent the structure of the input. The Abstract Syntax Tree provides a simplified representation by removing unnecessary grammatical details and retaining the essential

relationships between operators and operands.

The graphical trees use nodes and connecting lines to represent hierarchical relationships. This makes it possible to identify parent nodes, child nodes, and the organization of arithmetic expressions. Scrollbars help users navigate the tree when its dimensions exceed the visible area.

Comparing the two trees helps demonstrate the purpose of the AST. Users can observe how a detailed grammatical representation is reduced to a more compact structure suitable for further compiler processing.

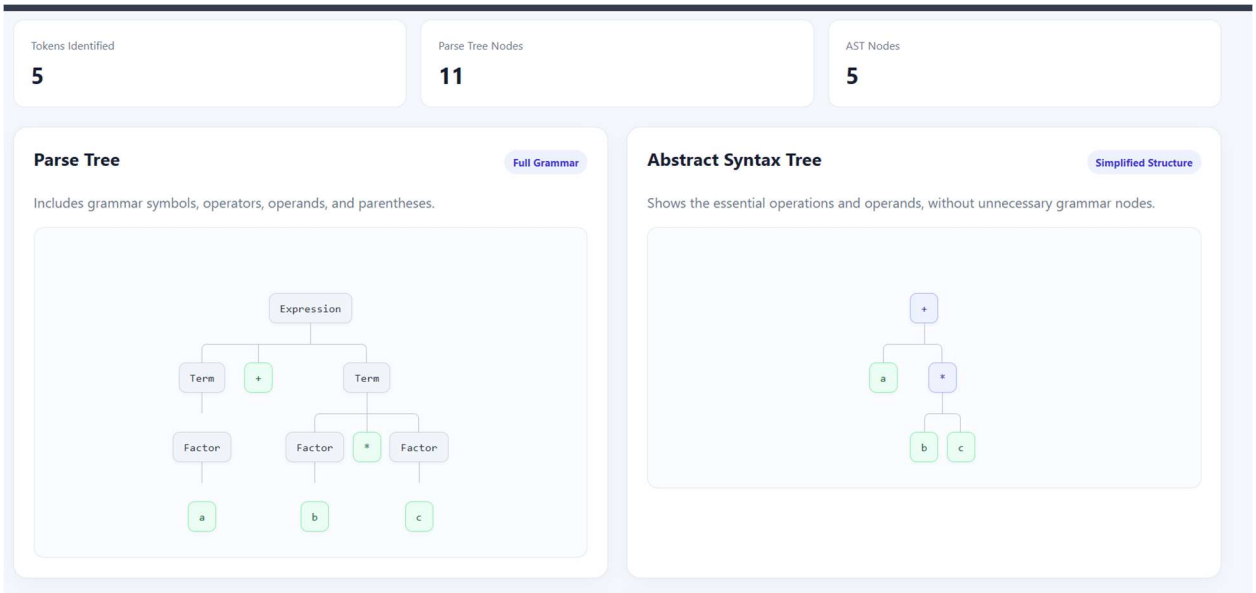


Figure 6.2: Graphical Representation of CST and AST

6.3 Token Information and Textual Tree Representation

The application provides additional sections to help users examine the processing results in detail. The Lexical Analysis section displays the tokens identified during lexical analysis. Each token is shown with its type and its value.

This information helps users understand how an expression is divided into individual elements before syntax analysis begins. It also makes it easier to identify the operators, identifiers, and numeric values recognized by the lexical analyzer.

The ASCII Trees section presents textual representations of the Concrete Syntax Tree and Abstract Syntax Tree. These representations show the hierarchical arrangement of the nodes using text and connecting characters. They provide an alternative way to inspect the tree structures without relying entirely on graphical visualization.

The application also displays a statistics summary containing the number of tokens, the number of nodes in the Concrete Syntax Tree, and the number of nodes in the Abstract Syntax Tree. This information illustrates how the Abstract Syntax Tree simplifies the Concrete Syntax Tree. Together, the token list, ASCII representations, and statistics summary provide a more detailed understanding of the expression-processing process.

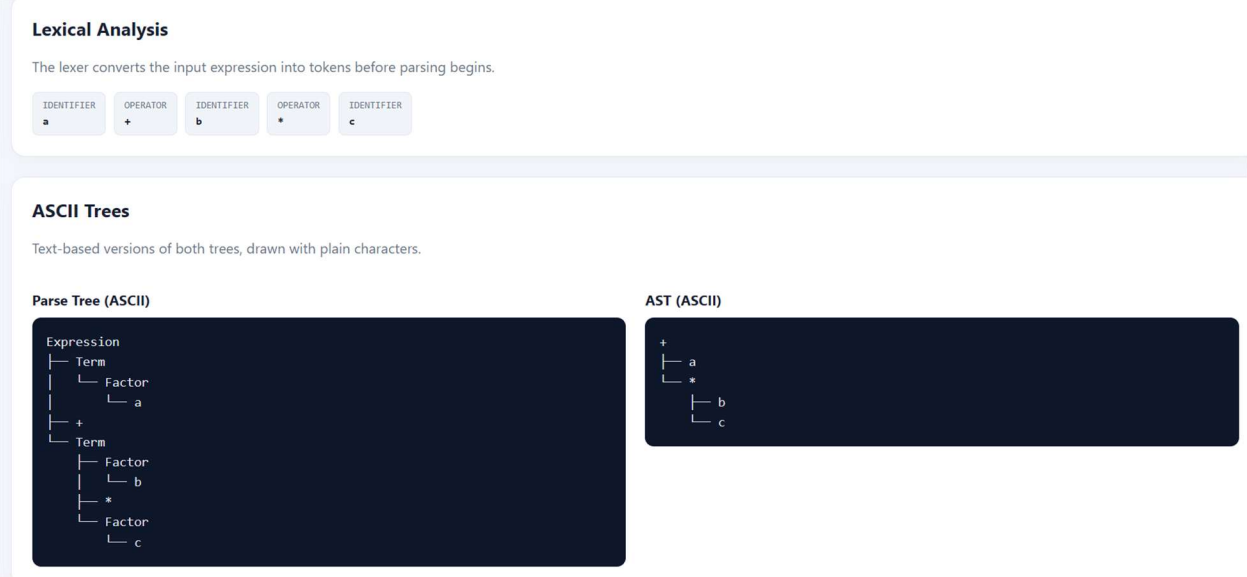


Figure 6.3: Token Information and ASCII Tree Representation

Table 1 : Lexical Analysis

Sr. No.	Token Type	Token Value
1	OPERATOR	(
2	IDENTIFIER	a
3	OPERATOR	+
4	IDENTIFIER	b
5	OPERATOR	)
6	OPERATOR	*
7	IDENTIFIER	c

CHAPTER 7

RESULTS AND CONCLUSION

7.1 Results

The project successfully demonstrates the fundamental stages of compiler design through an interactive Concrete Syntax Tree and Abstract Syntax Tree visualizer. It provides a web-based environment in which users can enter arithmetic expressions, analyze their tokens, and observe the structure of the parsed expression.

The lexical analysis component identifies different types of tokens, including identifiers, numeric literals, and operators, including parentheses. It also detects unsupported characters and reports lexical errors with their positions. This process helps demonstrate how a compiler converts a source expression into a sequence of meaningful tokens.

The syntax analysis component uses a recursive descent parsing technique to verify whether the input follows the supported grammar. It recognizes arithmetic expressions while respecting operator precedence. Multiplication and division are processed with higher precedence than addition and subtraction, and parentheses are used to control the grouping of expressions. Unary minus is also supported.

The system generates a Concrete Syntax Tree that represents the grammatical structure of the input and an Abstract Syntax Tree that provides a simplified representation of the essential operations. Both trees can be displayed graphically, allowing users to compare their structures and understand how syntax analysis contributes to the representation of a program.

The application also presents token information and provides a textual tree representation. These features offer different ways to examine the analysis results. The summary information, including status messages and node counts, helps users understand whether the input was processed successfully.

Overall, the project provides a useful educational demonstration of lexical analysis, syntax analysis, tree construction, operator precedence, and error reporting. Its web interface makes these compiler concepts easier to observe and understand.

7.2 Conclusion

The Concrete Syntax Tree and Abstract Syntax Tree Visualizer project demonstrates important concepts of compiler design through a simple and interactive application. It combines lexical analysis, recursive descent parsing, syntax tree generation, and graphical visualization in a single

system.

The project explains the transformation of an input expression into tokens, the verification of its grammatical structure, and the generation of tree representations. By displaying both the Concrete Syntax Tree and the Abstract Syntax Tree, the application highlights the difference between a detailed grammatical representation and a simplified structural representation of an expression.

The use of Python, Flask, and a browser interface makes the application suitable for educational purposes. The page allows users to enter expressions, view tokens, inspect tree structures, and understand parsing results without requiring a complex development environment. The inclusion of error reporting also helps users identify invalid input and understand the role of error detection during compilation.

Although the application demonstrates several essential compiler concepts, its supported grammar is limited to basic arithmetic expressions with parentheses and unary minus. It does not support assignment statements, and it does not implement every stage of a complete compiler, such as semantic analysis, intermediate code generation, code optimization, or machine code generation.

In conclusion, the project achieves its main objective of providing a visual and practical introduction to lexical analysis, syntax analysis, and syntax tree construction. It can serve as a foundation for further development, including support for additional language constructs such as assignments, improved error recovery, semantic analysis, and intermediate code generation.

APPENDIX

Appendix A – Test Cases and Results

The application was tested with valid and invalid expressions. The results are summarized in the tables below.

Table A.1: Valid expressions

Sr. No.	Input Expression	Tokens	CST Nodes	AST Nodes	Result
1	a + b * c	5	11	5	Success
2	(a + b) * c	7	16	5	Success
3	-a + b	4	10	4	Success
4	x / y - z	5	11	5	Success
5	10 + 20 * 3	5	11	5	Success

Table A.2: Invalid expressions

Sr. No.	Input Expression	Error Message Displayed
1	(empty input)	Please enter an expression.
2	a $ b	Invalid character at position 3: '$'
3	(a + b	Expected ')', found 'EOF'
4	a + * b	Unexpected token '*'. Expected a number, identifier, or '('
5	a + b)	Unexpected token ')'

Sample output for a + b * c

Abstract Syntax Tree:

```
+
├── a
└── *
    ├── b
    └── c
```

Concrete Syntax Tree:

Expression

```
├── Term
│   └── Factor
│       └── a
├── +
└── Term
    ├── Factor
    │   └── b
    ├── *
    └── Factor
        └── c
```

The AST has 5 nodes while the CST has 11. Multiplication appears below addition in the AST, which shows that operator precedence is preserved.

Sample output for (a + b) * c

Abstract Syntax Tree:

```
*
├── +
│   ├── a
│   └── b
└── c
```

The parentheses do not appear in the AST, but their effect is kept: addition is evaluated before multiplication.

Appendix B – Steps to Run the Project

1. Install Python 3 on the computer.
2. Install Flask by running pip install flask in a terminal.
3. Create a project folder with this structure:

```
parse_project/
├── app.py
└── templates/
    └── index.html
```

4. Open a terminal in the project folder and run python app.py.
5. Open a web browser and go to http://127.0.0.1:5000.
6. Enter an expression and click **Generate Trees**, or press **Ctrl+Enter**.